Consider the following Schema for answering the queries.

Product(maker, **model**, type)
PC(**model**, speed, ram, hd, price)
Laptop(**model**, speed, ram, hd, screen, price)
Printer(**model**, color, type, price)

The Product relation gives the manufacturer, model number and type (PC, laptop, or printer) of various products. We assume for convenience that model numbers are unique over all manufacturers and product types; that assumption is not realistic, and a real database would include a code for the manufacturer as part of the model number. The PC relation gives for each model number that is a PC the speed (of the processor, in gigahertz), the amount of RAM (in megabytes), the size of the hard disk (in gigabytes), and the price. The Laptop relation is similar, except that the screen size (in inches) is also included. The Printer relation records for each printer model whether the printer produces color output (true, if so), the process type (laser or ink-jet, typically), and the price. Identify all the correct options for natural language queries.

Find the manufacturer(s) of the computer (PC or laptop) with the highest available speed. 1/1

- [ ] pi maker(sigma speed=maxS((rho maxS(gamma ; MIN(speed)->maxS(pi model,speed (PC) union pi model, speed (Laptop)))) x ((sigma type='PC'(Product) join Product.model=PC.model pi model, speed (PC)) union (sigma type='laptop'(Product) join Product.model=Laptop.model pi model, speed (Laptop)))))
- [x] pi maker(sigma speed=maxS((rho maxS(gamma ; MAX(speed)->maxS(pi model, speed (PC) union pi model, speed (Laptop)))) x ((sigma type='PC'(Product) join Product.model=PC.model pi model, speed (PC)) union (sigma type='laptop'(Product) join Product.model=Laptop.model pi model, speed (Laptop)))))
- [ ] pi maker(pi maker,speed((sigma type='PC'(Product) join Product.model=PC.model pi model, speed (PC)) union (sigma type='laptop'(Product) join Product.model=Laptop.model pi model,speed (Laptop))) - pi X.maker,X.speed(sigma X.speed<Y.speed(rho X(pi maker,speed((sigma type='PC'(Product) join Product.model=PC.model PC) union (sigma type='laptop'(Product) join Product.model=Laptop.model Laptop))) x rho Y(pi maker,speed((sigma type='PC'(Product) join Product.model=PC.model pi model, speed (PC)) union (sigma type='laptop'(Product) join Product.model=Laptop.model pi model, speed (Laptop))))))))
- [x] pi maker(sigma speed>=maxS((rho maxS(gamma ; MAX(speed)->maxS(pi model, speed (PC) union pi model, speed (Laptop)))) x ((sigma type='PC'(Product) join Product.model=PC.model pi model, speed (PC)) union (sigma type='laptop'(Product) join Product.model=Laptop.model pi model,speed (Laptop)))))

Find those hard-disk sizes that occur in two or more PC's. 1/1

- [x] pi hd (sigma c >= 2 (gamma hd; count(model) -->c (PC)))
- [x] pi P1.hd(sigma P1.hd=PC.hd and P1.model != PC.model(rho P1(PC) cross join PC))
- [ ] pi P1.hd(sigma P1.hd=PC.hd (rho P1(PC) cross join PC))
- [ ] pi hd (sigma c >= 2 (gamma hd, model; count(model) -->c (PC)))

Find the manufacturers of PC's with at least three different speeds. Assume R = (sigma type='PC' (Product)) natural join PC 1/1

- [x] pi maker (sigma k>=3(gamma maker; COUNT(speed)->k (pi maker, speed (R))))
- [ ] pi maker (sigma k>=3(gamma maker; COUNT(speed)->k (R)))
- [x] pi R1.maker(sigma R1.maker=R2.maker and R1.maker=R3.maker and R1.speed!=R2.speed and R1.speed!=R3.speed and R2.speed!=R3.speed and R1.model!=R2.model and R1.model!=R3.model and R2.model!=R3.model (rho R1 (R) x rho R2 (R) x rho R3(R)))
- [ ] pi maker(sigma c>=3 (gamma maker, speed; COUNT(model)->c (R)))

Find those pairs of PC models that have both the same speed and RAM. A pair should be listed only once; e.g., list (i,j) but not (j,i). 1/1

- [x] pi P1.model,P2.model(sigma P1.speed=P2.speed and P1.ram=P2.ram and P1.model<P2.model(rho P1(PC) x rho P2(PC)))
- [ ] pi P1.model,P2.model(sigma P1.speed=P2.speed and P1.ram=P2.ram and P1.model!=P2.model(rho P1(PC) x rho P2(PC)))
- [ ] pi P1.model,P2.model(sigma P1.speed=P2.speed or P1.ram=P2.ram and P1.model<P2.model(rho P1(PC) x rho P2(PC)))
- [ ] pi P1.model,P2.model(sigma P1.speed=P2.speed and P1.ram=P2.ram and P1.model<P2.model(PC x PC))